📄 Product Requirements Document

TickVent HB — Mobile Event Ticketing App

---

1. 📌 Overview

TickVent HB is a mobile application that enables users to:

- Discover events
- View event details
- Purchase tickets
- Bookmark/save favorite events

The system integrates with a REST API (JSON-based) and uses local persistence (Room) for offline features like bookmarks.

---

2. 🎯 Objectives

- Provide seamless event discovery experience
- Enable fast and secure ticket purchasing
- Ensure reactive UI with real-time data updates
- Maintain offline capabilities (favorites/bookmarks)

---

3. 👤 Target Users

- Students and young adults attending events
- Concert, festival, and exhibition attendees
- Users who frequently browse events

---

4. 🧱 System Architecture

4.1 Architectural Pattern

- **MVVM (Model-View-ViewModel)**
- Clear **Separation of Concerns**

4.2 Layers

UI Layer

- Activities / Fragments
- RecyclerView + Adapter
- Observes LiveData / StateFlow

ViewModel Layer

- Handles UI logic
- Exposes reactive state (LiveData / StateFlow)
- Communicates with Repository

Data Layer

- Repository Pattern
- Remote: Retrofit API
- Local: Room Database

---

5. 📊 Data Model Mapping (Based on Provided Schema)

Core Entities Used:

- **Events**
- **Venues**
- **Attractions**
- **Images**
- **Offers**
- **Prices / Price-Zones**
- **Genres / Segments**

Example Simplified Kotlin Models

```
data class Event(
  val id: String,
  val name: String,
  val type: String,
  val dates: String,
  val venue: Venue,
```

```
  val images: List<Image>,
  val priceRange: PriceRange?
)

data class Venue(
  val name: String,
  val city: String,
  val country: String
)

data class Image(
  val url: String,
  val width: Int,
  val height: Int
)
```

---

6. 📱 Core Features

6.1 Event Browsing

- List of events using RecyclerView
- Pagination or lazy loading
- Filter by:
    - Genre
    - Location
    - Date

6.2 Event Details

- Event information:
    - Name
    - Venue
    - Date
    - Images
    - Price range
- “Buy Ticket” CTA
- Bookmark button

6.3 Ticket Purchase

- Select price zone
- View ticket availability
- Confirm purchase

6.4 Favorites / Bookmarks (Room)

- Save event locally
- Offline access
- Sync with UI reactively

---

7. 📱 Screens (Minimum 3 Required)

7.1 Home Screen (Event List)

Components:

- RecyclerView (Event list)
- Search bar
- Filter options

RecyclerView Item:

- Event image
- Name
- Date
- Location

Behavior:

- Fetch data from API
- Observe ViewModel state
- Auto-update UI

7.2 Event Detail Screen

Components:

- Image carousel (RecyclerView horizontal)
- Event info

- Price range
- Buy button
- Bookmark toggle

7.3 Favorites Screen

Components:

- RecyclerView (saved events)
- Empty state UI

Data Source:

- Room database

---

8. 🔄 State Management

Use:

- **LiveData** OR **StateFlow (preferred for scalability)**

Example UI State

```
sealed class UiState<out T> {
  object Loading : UiState<Nothing>()
  data class Success<T>(val data: T) : UiState<T>()
  data class Error(val message: String) : UiState<Nothing>()
  object Empty : UiState<Nothing>()
}
```

Reactive Flow

- API → Repository → ViewModel → UI
- Room → Flow → ViewModel → UI

---

9. 🌐 Networking (Retrofit)

Requirements:

- REST API consumption
- JSON parsing (Gson / Moshi)

Example:

```
interface ApiService {
  @GET("events")
  suspend fun getEvents(): Response<EventResponse>
}
```

---

10. 💾 Local Storage (Room)

Use Cases:

- Save bookmarks
- Offline viewing

Example Entity:

```
@Entity(tableName = "favorites")
data class FavoriteEvent(
  @PrimaryKey val id: String,
  val name: String,
  val imageUrl: String
)
```

---

11. 🔁 Repository Pattern

```
class EventRepository(
  private val api: ApiService,
  private val dao: FavoriteDao
) {
  suspend fun getEvents() = api.getEvents()

  fun getFavorites() = dao.getAllFavorites()

  suspend fun saveFavorite(event: FavoriteEvent) = dao.insert(event)
}
```

---

12. 📋 RecyclerView Implementation

Requirements:

- Must use Adapter pattern
- Efficient ViewHolder pattern

Example:

```
class EventAdapter : RecyclerView.Adapter<EventViewHolder>() {
  private val items = mutableListOf<Event>()

  fun submitList(data: List<Event>) {
    items.clear()
    items.addAll(data)
    notifyDataSetChanged()
  }
}
```

---

13. ⚠️ Error Handling

Cases:

1. API Failure

- Show error message
- Retry button

2. Empty Data

- Show “No events available”

3. Loading State

- Show shimmer / progress bar

---

14. 🚫 UI Constraints

- ❌ No manual LinearLayout
- ✅ Use ConstraintLayout or modern layouts
- ✅ RecyclerView for lists

---

15. 🔄 UI Reactivity

- UI must automatically update when:
  - API data changes
  - Favorites updated
- Observed via LiveData / StateFlow

---

16. 🔐 Non-Functional Requirements

- Smooth scrolling (RecyclerView optimization)
- Efficient memory usage (image loading with Glide/Coil)
- API timeout handling
- Offline fallback for bookmarks

---

17. 📈 Future Enhancements

- Authentication (login/signup)
- Payment gateway integration
- Push notifications
- Personalized recommendations

---

18. 🧩 Suggested Tech Stack

- Kotlin
- MVVM
- Retrofit
- Room
- StateFlow (preferred)
- Glide / Coil
- Coroutines

---

19. 🔁 End-to-End Flow

1. User opens app → Home screen loads

2. ViewModel triggers API fetch
3. UI observes state:
    - Loading → show loader
    - Success → display RecyclerView
4. User clicks event → Detail screen
5. User bookmarks → Room updated → UI auto refresh